Configuration Management System for Kane AI & Test Manager.

A system to efficiently manage and allocate test configurations across various environments within KaneAI and Test Manager.

Team: Product Developement
Contributor: Mridul Shukla
Status: Ready for Review
Launching on: TBD (post-development timeline)
Resources: [https://www.lambdatest.com/test-manager, https://www.lambdatest.com/blog/introducing-kane-ai]

Problem Definition:

Managing test configurations across desktops, real devices, and virtual devices is messy and prone to errors. QA engineers and development managers often struggle to assign the right configurations to test cases and test runs. This leads to misconfigurations, delays, and more time spent on maintenance than necessary. It's becoming harder to scale the testing process reliably with the current setup.

We need a solid **Configuration Management System** to streamline this process and reduce human errors.

With it, assigning the right configurations will be easier, faster, and more accurate. Solving this now is key to keeping our testing infrastructure scalable, efficient, and reliable.

Considerations	Brief
What is the problem?	Managing test configurations is messy and prone to errors, making it hard to scale the testing process.
Who is facing the problem?	QA engineers and development managers struggle with assigning the right configurations to test cases.

What is the business value unlocked by solving this problem?	Reducing misconfigurations, speeding up the allocation process, and improving the reliability of testing.
How will target users benefit if the problem is solved?	QA engineers and dev managers will spend less time on configuration management and more on testing and development.
Why is it urgent to solve this problem now?	The current setup is inefficient, increasing maintenance time and making scaling the testing infrastructure harder.

Goals:

High-Level Goals (in Priority Order):

1. **Increase efficiency in assigning configurations** to test cases and test runs, reducing misconfigurations and delays.

2. **Reduce manual intervention and human error** in managing test configurations, enabling smoother test execution.

3. **Scale the testing infrastructure** reliably by automating configuration allocation and improving configuration management.

Functional metrics to track	Non-functional metrics to track
1. **L0- Efficiency**: Time taken to assign configurations.	**User effort**: Time taken by the user to assign configurations.
2. **L1- Configuration Allocation Success Rate**: Percentage of successful configuration allocations.	**Accuracy**: Success rate of configuration assignments.
3. **L2- Automation Level**: Percentage of configurations assigned via automation.	**System performance**: Ability to handle increased number of test cases and configurations.
4. **L3- Reduction in errors**: Number of errors caused by misconfigurations.	**Stability**: Reliability of the system in real-time configuration updates.

North star metric: Time saved per test case due to improved configuration assignment

Importance of Metrics:

- **Efficiency metrics** measure how quickly and effectively configurations are assigned, reducing delays and misconfigurations.
- **Success rate and automation level** ensure smoother and more consistent configuration management.
- **Error reduction metrics** ensure higher accuracy and fewer issues during test execution.
- **Non-functional metrics** focus on scalability and system reliability, ensuring smooth performance even as the test cases grow.

Non-Goals:

- Developing a new test execution framework.
- Overhauling existing test case management systems.
- Managing infrastructure provisioning for test environments.

Validation of the problem:

Feedback from QA teams indicates frequent challenges in managing test configurations, leading to delays and errors in test execution. Competitor analysis reveals that streamlined configuration management significantly enhances testing efficiency.

Understanding the target audience:

User Segment in Focus: QA Engineers, Development Managers, and Test Engineers who work with KaneAI and Test Manager to ensure configurations are correctly allocated and maintained for testing purposes.

Key Personas:

Key Persona	**Sheena** (QA Engineer) (3+ years experience, eCommerce company)	**John** (Development Manager) (10+ years experience, SaaS company)
Pain Points	● Struggles with assigning test configurations quickly. ● Faces errors and delays due to misconfigured environments.	● Difficult to track and update configurations across multiple test runs ● Needs consistency across configurations for team-wide use
Unmet Needs	● Needs faster and error-free configuration assignment. ● Needs better visibility and management of configuration updates.	● Needs more efficient and transparent configuration management across teams. ● Seeks integration with development tools for seamless collaboration.

Possible Solutions:

1. **Automated Configuration Allocation**
    - **Features**:
        - AI-driven automation for test configuration assignment.

        - Intelligent recommendations for device/browser configurations based on previous test data and requirements.
        - Support for a variety of configuration types (Desktop, Real Device, Virtual Device).
        - Regex matching for Manufacturer, Device Name, and OS versions.
    - **Technology**:
        - Machine learning models for configuration prediction and selection.
        - Integration with existing test management systems like Test Manager for seamless workflow.
    - **User Feedback Integration**:
        - Collect user feedback on configuration accuracy to improve future allocation through AI learning.

2. **Configuration Error Reduction**
    - **Features**:
        - Real-time error checking and alerts for misconfigurations during the allocation process.
        - Ability to quickly spot and correct misconfigurations before tests run.
    - **Technology**:
        - Validation algorithms that cross-check allocated configurations against the test cases.
        - Integration with Test Manager to provide instant feedback.

3. **Version Management for Applications**
    - **Features**:
        - Easy version management for applications associated with real devices and virtual environments.
        - Streamlined process for updating and managing new application versions.

- **Technology**:
    - Cloud-based version control and auto-updating of app configurations.

Chosen Solution: Automated Configuration Allocation

- **Reasoning**:
    - Directly addresses the main frustration of misconfigurations and inefficiency in the allocation process.
    - Aligns with the need for greater scalability and speed in handling increasing test environments.
    - Reduces manual intervention, enhancing reliability and speed.

Evaluation Parameters:

- **User Impact**: Addresses the primary frustration of misallocation and configuration errors.
- **Ease of Implementation**: Feasible through AI-driven solutions and integration with existing systems.
- **Cost-effectiveness**: Makes use of existing test management infrastructure and AI-driven automation, reducing manual overhead.
- **Retention Potential**: Ensures better scalability and more reliable test execution, leading to fewer errors and delays in test cycles.

Solution Overview:

High-Level Solution:

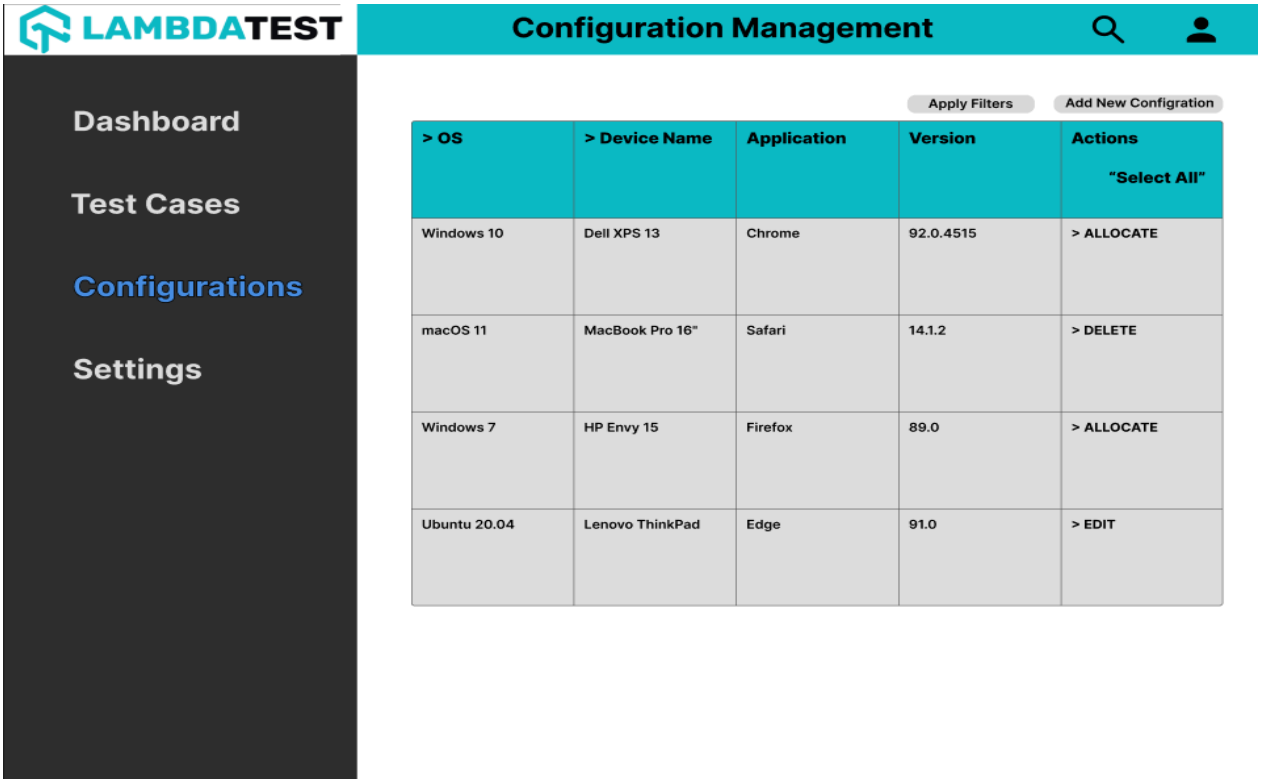
Improve the configuration management process through AI-driven automation and intelligent allocation. This solution streamlines configuration assignment, ensures better accuracy, and reduces the time spent on manual configuration setup.

User Flow:(Expanded View)

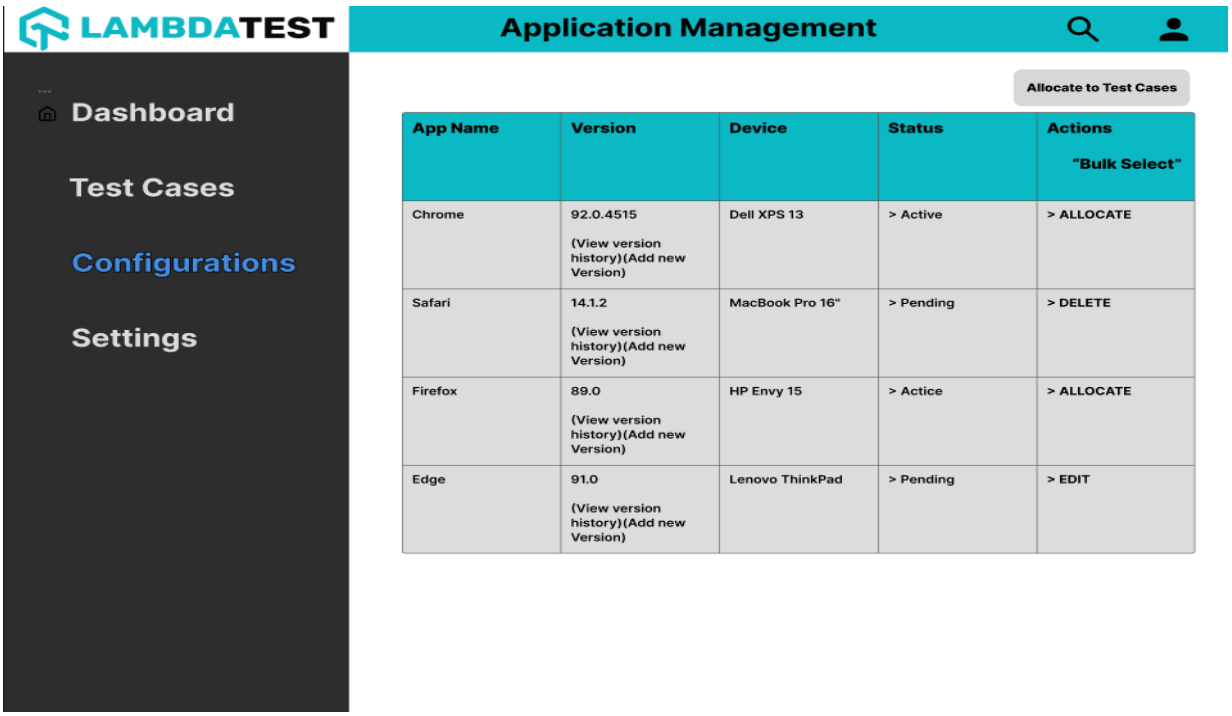


Wireframes:

Configuration Management Interface(Lo-fi):



Application Management Interface(Lo-fi):



Key Features:

Key Features	Brief
Personalized Configuration Allocation	Auto-assigns configurations based on user behavior (device, OS).
Test Run Allocation	Allocates configurations to test runs for efficient execution.
Advanced Configuration Filters	Filter configurations by OS, device, browser, etc.
Bulk Allocation	Assign configurations to multiple test cases at once.
Regex Support for Real Devices	Filters real device parameters using regular expressions.

Launch Readiness:

Steps leading to launch	Brief
Design complete	Finalize wireframes and UI/UX
Development	Build AI-driven automated configuration allocation system, integrate regex matching, and ensure version control for applications.
Testing	Unit testing → QA testing → Scalability & stability checks
Dogfooding	Internal testing and feedback iteration
Launch Prep	Stakeholders reviews, support team training, user documentation
Go-live	Phased roll-out with performance monitoring

Key Milestones:

1. **Design Complete:** [Date]
2. **Development Complete:** [Date]
3. **QA Testing:** [Date Range]
4. **Dogfooding:** [Date Range]
5. **Launch Readiness Review:** [Date]
6. **Launch:** [Go-Live Date]

Launch Checklist:

- **Support:** Training and updated escalation processes for handling configuration issues.
- **Operations:** Monitoring dashboards set up for tracking configuration allocations and application updates.
- **Documentation:** User guides, FAQs, and stakeholder reports for seamless onboarding.
- **Stakeholders:** Product, Engineering, QA, Marketing, and Support Teams.
- **Post-Launch:** Real-time performance and engagement tracking for configuration management.

Experimentation Plan:

- **A/B Testing:** Compare AI-driven configuration allocation recommendations with the baseline.
- **Metrics:** Configuration allocation success rate, time saved per test case, error reduction, system performance under load.

Open Questions & Decisions Taken:

- **Should configuration allocation errors impact the test execution flow or be handled independently?**

- **How to ensure scalability while maintaining configuration accuracy across growing test environments?**

- **How frequently should application versions be updated across different test configurations?**

- **Feedback loop integration: How will user feedback be incorporated into AI learning for configuration predictions?**